# Vision-Based Connect-4 Robot: Closed-Loop Game-State Extraction and Verified Token Placement


Andrew Abdelmalak
*Dept. of Mechatronics Engineering*
*German University in Cairo*
Cairo, Egypt
55-22771

Daniel Boules
*Dept. of Mechatronics Engineering*
*German University in Cairo*
Cairo, Egypt
55-5055

David Girgis
*Dept. of Mechatronics Engineering*
*German University in Cairo*
Cairo, Egypt
55-1481

Kirolous Kirolous
*Dept. of Mechatronics Engineering*
*German University in Cairo*
Cairo, Egypt
55-18081

Samir Yacoub
*Dept. of Mechatronics Engineering*
*German University in Cairo*
Cairo, Egypt
55-25111

Youssef Salama
*Dept. of Mechatronics Engineering*
*German University in Cairo*
Cairo, Egypt
55-0540



***Abstract*—This paper presents a full vision-guided Connect-4 robot that couples board-state extraction, trusted-state validation, minimax move planning, physical token dispensing, and post-move camera verification. A Raspberry Pi camera module in front of the board observes it while a Raspberry Pi 4 [1] runs a Python/OpenCV [2] pipeline that localizes the blue frame, warps the board to an $800 \times 800$ view, conditions the image, and classifies each cell in one of three modes: HSV, Auto, or ML. The accepted state is passed through a six-rule validator before a depth-5 minimax solver selects the robot move. The target column is then sent at 9600 baud to an Arduino-controlled carriage and magazine mechanism, and the camera re-reads the board to confirm that the expected token actually appeared. Across approximately 200 labeled cell ROIs in the final evaluation set, HSV achieved 98% per-cell accuracy, Auto achieved 96%, and ML achieved 91%, making HSV the strongest practical mode under the tested indoor setup. The drop mechanism achieved 95% overall success, with 100% success in columns 1–3 and 85% in column 7, showing that open-loop carriage travel, not segmentation, was the dominant physical bottleneck. The reported end-to-end cycle averaged 7.5 s, and the logic layer was checked separately by automated tests. These results document representative end-to-end closed-loop demonstrations backed by separate logic-layer validation, while remaining limited by lighting shift, domain shift, and dispenser mechanics.**



***Index Terms*—Connect-4 robot, computer vision, HSV segmentation, trusted-state validation, closed-loop verification**


## I. Introduction

Autonomous board-game robots are useful mechatronic benchmarks because they force perception, reasoning, and actuation to agree on the same physical state in real time [3]. A Connect-4 robot must read the board, infer the next legal move, position a dispenser over the chosen column, and verify that the physical outcome matches the internal plan. The challenge is not only to solve each stage independently, but to prevent small perception or actuation errors from silently corrupting the game state.

Prior work has treated the main subproblems separately. Vision papers show that canonical board rectification and per-cell inference are effective for structured tabletop games [4], [5]. HSV-based color segmentation remains attractive in robotic handling when illumination is reasonably controlled and the target colors are well separated [6]. Chess-playing robots also show that robust physical manipulation of game pieces is often harder than the game reasoning itself [7]. Connect-4 simplifies the manipulator geometry because only lateral positioning is needed, but it still requires robust token-color recognition and gravity-aware validation.

The gap addressed in this project is the lack of a tightly documented end-to-end Connect-4 implementation in which camera-based perception, trusted board-state acceptance, physical token placement, and post-move re-verification are all treated as one closed loop. For course deliverables, it is easy to report a working classifier, a working minimax solver, or a working dispenser separately. The harder and more important question is whether the complete system keeps a trustworthy state when the real hardware misses, stalls, or sees ambiguous frames.

This paper makes four concrete contributions.

1) It documents a complete vision-guided Connect-4 robot in which a trusted-state validator gates the board before action and a camera re-check gates the board after action.
2) It benchmarks three perception modes – HSV, Auto, and ML – and shows that HSV is the strongest practical operating mode under the tested indoor conditions.
3) It shows that the dominant real-world bottleneck is mechanical drop reliability, especially for long carriage

travel toward the far columns, rather than image segmentation accuracy.

4) It reports representative end-to-end demonstrations with honest limits: the integrated robot closes the loop in the archived demos, while separate logic checks show that the state machine remained stable before hardware uncertainty entered the loop.

The remainder of the paper is organized as follows. Section II reviews the most relevant literature and motivates the system boundary used here. Section III summarizes the integrated architecture and the final physical setup. Section IV explains the vision and state-estimation pipeline and compares the three classifier modes. Section V describes trusted-state validation, move planning, and post-drop verification. Section VI interprets the final experimental results. Section VII discusses the main lessons and limitations. Section VIII concludes.

## II. Related Work

### A. Board-State Extraction

Wolflein and Arandjelovic decomposed chess-state extraction into board localization, homographic rectification, and square-level recognition [4]. That decomposition transfers well to Connect-4 because the board is still a fixed grid, but the classifier changes from piece identity to three states: empty, red, and yellow. Masouris and van Gemert later showed that end-to-end learning can reduce hand-crafted geometry stages at the cost of a larger training burden [5]. For a course project with modest data, the modular route is more practical because each failure can be inspected directly.

### B. Color Segmentation Under Lighting Variation

Akış and Dişlitaş showed that HSV-based object detection remains strong in robotic handling when color is the dominant cue and lighting is managed carefully [6]. Their findings match the Connect-4 setting: token hue is informative, while raw RGB intensity alone is less stable. The same literature also warns that fixed thresholds weaken under large lighting shifts, which matters here because the board and tokens are glossy plastic surfaces.

### C. Game Robots and Embedded Decision Layers

Board-game robots such as Gambit show that unconstrained manipulation of pieces, not game reasoning, is often the harder part of the problem [7]. On the decision side, depth-limited minimax with alpha-beta pruning remains a practical baseline for Connect-4 because the game is structured, discrete, and fully observable [8]. The missing piece is the interface between these subsystems: a robot still needs a trusted physical board state before the search is meaningful, and a verified post-move state before the next turn is safe.

## III. System Overview

Figure 1 summarizes the system as a closed loop with two state gates. The first gate is trusted-state validation before planning. The second gate is post-drop camera verification before the internal state advances to the next turn.

The final setup used a Raspberry Pi 4 [1] for perception and planning, an Arduino Uno [9] for motor control, a Raspberry Pi camera module facing the board, a time-based lateral carriage, and an encoder-coupled magazine release. The final as-built hardware cost reported in the course materials was 6100 EGP. The operator dashboard ran on the Pi and exposed the raw feed, the warped feed, board-state views, classifier mode selection, and robot status.

TABLE I
Per-cell classification accuracy by operating mode

| Mode | Empty | Red | Yellow | Overall |
|---|---|---|---|---|
| HSV | 99% | 98% | 98% | 98% |
| Auto | 97% | 95% | 96% | 96% |
| ML | 94% | 92% | 88% | 91% |

## IV. Vision and State Estimation

The perception stack starts from a single front-facing image of the board and converts it into a board state that the robot can trust enough to plan from. The board is first localized from the blue frame, then rectified to a canonical top-down view using the homography in (1)

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{1}$$

After warping, the deployed runtime applies a brightness shift of $c = +20$, a contrast gain of $\alpha = 1.3$, and a $5 \times 5$ Gaussian blur before partitioning the board into a $6 \times 7$ lattice. These are the values in the deployed runtime included with the repository; the final course report lists an earlier operating point of $c = +30$ and $\alpha = 1.4$.

Figure 3 gives the curated pipeline story for one representative board. Each panel answers a different question: whether the raw scene is readable, whether geometry has been normalized, whether token positions are interpreted correctly, and whether the result is usable as a state matrix.

The system exposes three classifier modes.

- **HSV**: rule-based token classification using calibrated red and yellow HSV ranges.
- **ML**: a RandomForest classifier trained on compact color-statistics features extracted from cell regions.
- **Auto**: a hybrid mode that prefers the ML prediction when its confidence exceeds 0.65 and otherwise falls back to HSV.

Table I reports the final per-cell comparison across approximately 200 labeled cell ROIs captured under the deployment lighting. The table is useful not just because it ranks the modes, but because it shows where the weaker modes fail: the ML path drops most sharply on yellow cells, and the hybrid mode inherits some of that penalty even though it has access to the HSV fallback.

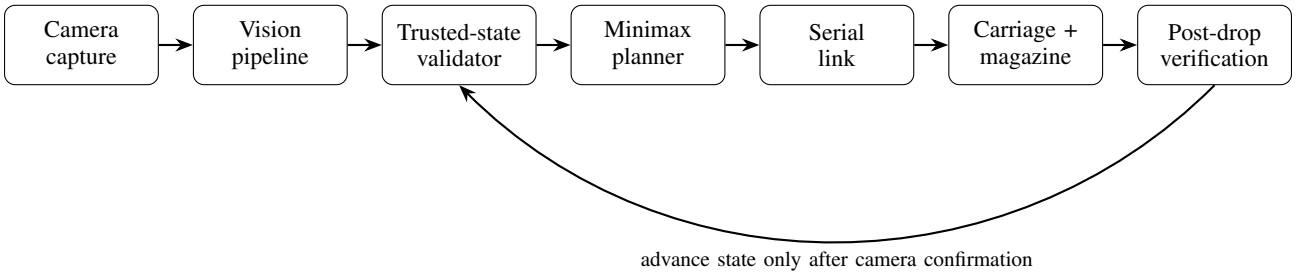



Fig. 1. Integrated closed-loop architecture. The camera is used twice: first to infer a candidate board and later to confirm that the physical robot move actually happened. This is the core difference between a working pipeline demo and a trustworthy integrated robot.

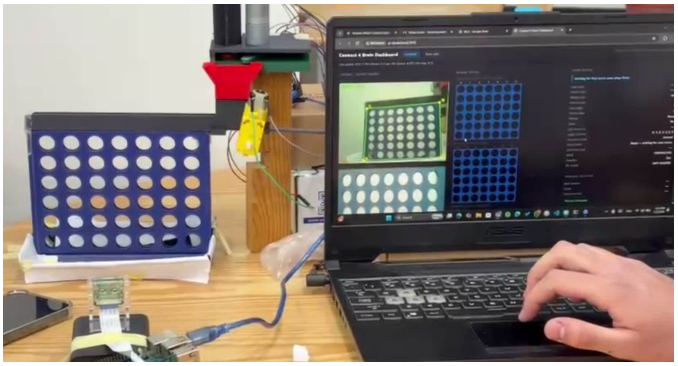


Fig. 2. Final assembled evaluation setup. The physical board, overhead dispenser, camera mount, Raspberry Pi, and operator dashboard are visible in one frame, showing the exact integrated system used in the final demonstrations.

## V. Closed-Loop Decision and Actuation

The board extracted from vision is not accepted blindly. Instead, it is compared against the last trusted board and must satisfy six rules before planning begins: the board must have changed, no chip may disappear, exactly one new chip may appear, the new chip must respect gravity, the target column must not already be full, and the new chip color must match the current turn. These checks are what keep imperfect frame-by-frame perception from becoming a corrupted game state.

Once a candidate board passes validation, the robot chooses a move using the depth-5 minimax recursion in (2) with alpha-beta pruning,

$$V(S) = \max_{a \in \mathcal{A}(S)} \min_{b \in \mathcal{A}(T(S,a))} V\big(T(T(S,a),b)\big), \qquad (2)$$

where $\mathcal{A}(S)$ is the set of legal columns and $T(S,a)$ is the gravity-respecting state transition induced by dropping a token in column $a$. The archived runtime also includes immediate-win and immediate-block shortcuts before the full search, which matters because the robot only needs a good move quickly, not a mathematically exhaustive search of every future board.

The chosen column is transmitted at 9600 baud to the Arduino. The carriage first moves laterally using calibrated durations $[0, 330, 600, 900, 1200, 1500, 1800]$ ms for columns 1 through 7 and holds 2 s over the target; the magazine motor then rotates 430 encoder pulses with a fast/slow two-stage profile to release one token, and 0.5 s later the carriage returns home. After a 2.5 s settling delay, the camera polls the board every 0.4 s until 10 s after the command. The internal state advances only if the expected new token is observed in the commanded column.

## VI. Experimental Results

### A. Why HSV Is the Practical Default

Table I shows that HSV reached 98% overall per-cell accuracy, ahead of Auto at 96% and ML at 91%. The number matters because it establishes the deployed operating point: under the tested indoor setup, the simplest classifier was also the best one. The reason is visible in the project conditions and code structure. The board frame and tokens occupy distinct hue ranges, while the ML path was trained on a much smaller labeled set and had to generalize from compact color-statistics features. In practice, that left the learned model more exposed to lighting and appearance variation, especially for yellow cells.

The Auto mode is still useful because it demonstrates an integrated bonus path rather than a disconnected side experiment, but its lower score shows that adding a learned branch is not automatically an upgrade. In this system, the hybrid only becomes meaningfully attractive if a larger and more diverse labeled dataset is collected.

### B. Mechanical Drop Reliability Dominates the Real Bottleneck

Figure 4 reports the final token-drop success rates over 20 trials per column. Columns 1–3 achieved 100% success, columns 4 and 5 achieved 95%, column 6 achieved 90%, and column 7 dropped to 85%. The overall success rate was 95% across 140 drops.

These numbers matter because they change the diagnosis of the system. The robot already sees the board well enough for play; what it does not yet do equally well is deliver the physical token to every column with the same repeatability. The reason is the time-based carriage. Travel error accumulates as the commanded distance grows, so the far columns are more exposed than the near ones. Once the carriage is correctly aligned, the encoder-coupled magazine release is much more

repeatable than the carriage itself. That is why the main reliability curve is indexed by column distance rather than by token color or board occupancy.

### C. Integrated End-to-End Behavior

The final materials reported an end-to-end cycle time from accepted board state to verified robot move ranging from 3 to 15 s, with a 7.5 s average. That number matters because it reflects what the user experiences, not just what the image-processing notebook can do in isolation. The reason for the gap between millisecond-scale perception and multi-second gameplay is that the full loop includes motion, settling time, and post-drop verification. Earlier benchmarking on the same project reported only 8.47 ms for rotation, 3.60 ms for perspective warp, and 0.76 ms for the deployed $5 \times 5$ Gaussian stage on the Pi, so the visible latency is dominated by mechanics and confirmation rather than by image processing itself.

Figure 5 shows the system-level proof sequence used in the final demo story. A human move is first accepted into the trusted state, the robot then plans its response, and the next accepted frame confirms the post-drop board. This matters because it demonstrates the actual closed loop, not just a sequence of isolated screenshots.

The same final course materials also recorded 31 of 31 passed core checks spanning win detection, gravity-correct updates, validator rules, minimax edge cases, and simulator progressions. That test file was not kept; the repository carries its own automated suite over the same logic. That result matters because it isolates the logic layer from the hardware layer. The reason it strengthens the overall project is that it shows the state machine and planner were stable before mechanical uncertainty entered the loop, even though it should not be read as a counted benchmark of full physical games.

## VII. Discussion and Limitations

The strongest outcome of this project is not any one classifier score or any one demo video. It is the fact that the robot treats both perception and actuation as potentially fallible and therefore refuses to advance its internal state without evidence. The trusted-state validator protects the planner from impossible boards, and the post-drop re-check protects the planner from believing a move happened when the hardware actually missed. In a physical robot, that distinction is more valuable than a marginal gain in standalone classifier accuracy.

The three-mode comparison also leads to a practical, not ideological, conclusion. HSV is not universally better than learning-based methods, but it is the right default for the tested setup because its assumptions match the data that was actually collected: a fixed camera, a known blue frame, red and yellow tokens, and controlled indoor lighting. The ML and Auto modes remain useful because they show an extensible path beyond thresholding, but the current evidence does not justify displacing HSV as the primary runtime mode.

The main limitation is the mechanical dispenser, especially the time-based carriage. The inner columns prove that the concept works, but the outer-column degradation shows that the current actuation path is not yet uniformly reliable across the full board width. A carriage encoder or homing feedback would attack the dominant failure directly. This is the highest-leverage hardware improvement because it addresses the part of the loop that the final data identifies as the actual bottleneck.

The second major limitation is environmental robustness. The final tests were run with a fixed mounting geometry, a stable indoor setup, and the blue board clearly separated from the background. That is a fair engineering boundary for a course robot, but it is not evidence of universal deployment robustness. Lighting shifts, glare, and domain changes remain real limits, especially for the fixed-threshold HSV path and the small-data ML path. The current system therefore demonstrates successful integrated operation under the tested conditions, not guaranteed generalization to arbitrary rooms or camera placements.

## VIII. Conclusion

This paper documented a complete vision-guided Connect-4 robot in which board reading, trusted-state acceptance, move planning, token placement, and camera-based re-verification form one closed loop. The final results support four clear conclusions: the project achieved representative end-to-end integrated demonstrations, HSV was the strongest practical vision mode under the tested setup, mechanical drop reliability was the dominant real-world bottleneck, and the final system remained limited by lighting shift and open-loop carriage mechanics.

The key lesson is therefore architectural as much as algorithmic. A modest mechatronics system becomes far more trustworthy when it treats camera inference and physical execution as hypotheses to be checked, not as facts to be assumed. That lesson generalizes beyond Connect-4 and is the main reason this project reaches a stronger final quality bar than a checklist-driven pipeline demo.


## Acknowledgment

The authors thank Dr. Omar M. Shehata for guiding MCTR 1010: Image Processing for Mechatronics at the German University in Cairo.


## References


[1] Raspberry Pi Foundation, "Raspberry pi 4 model B," https://www.raspberrypi.org/, 2024, accessed: 28 Jun 2026.

[2] OpenCV team, "OpenCV: Open source computer vision library," https://opencv.org/, 2024, accessed: 28 Jun 2026.

[3] A. Ayub and A. R. Wagner, "Teach me what you want to play: Learning variants of connect four through human–robot interaction," in *Social Robotics (ICSR 2020)*, ser. Lecture Notes in Computer Science. Springer, 2020, pp. 502–515.

[4] G. Wölflein and O. Arandjelović, "Determining chess game state from an image," *J. Imaging*, vol. 7, no. 6, p. 94, Jun. 2021.

[5] A. Masouris and J. van Gemert, "End-to-end chess recognition," *arXiv preprint arXiv:2310.04086*, 2023, arXiv:2310.04086 [cs.CV].

[6] U. Akış and S. Dişlitaş, "Intelligent lighting system using color-based image processing for object detection in robotic handling applications," *Appl. Sci.*, vol. 14, no. 7, p. 3002, Apr. 2024.

[7] C. Matuszek, B. Mayton, R. Aimi, M. P. Deisenroth, L. Bo, R. Chu, M. Kung, L. LeGrand, J. R. Smith, and D. Fox, "Gambit: An autonomous chess-playing robotic system," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, May 2011, pp. 4291–4297.

[8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
[9] Arduino, “Arduino UNO R3,” https://www.arduino.cc/, 2024, accessed: 28 Jun 2026.

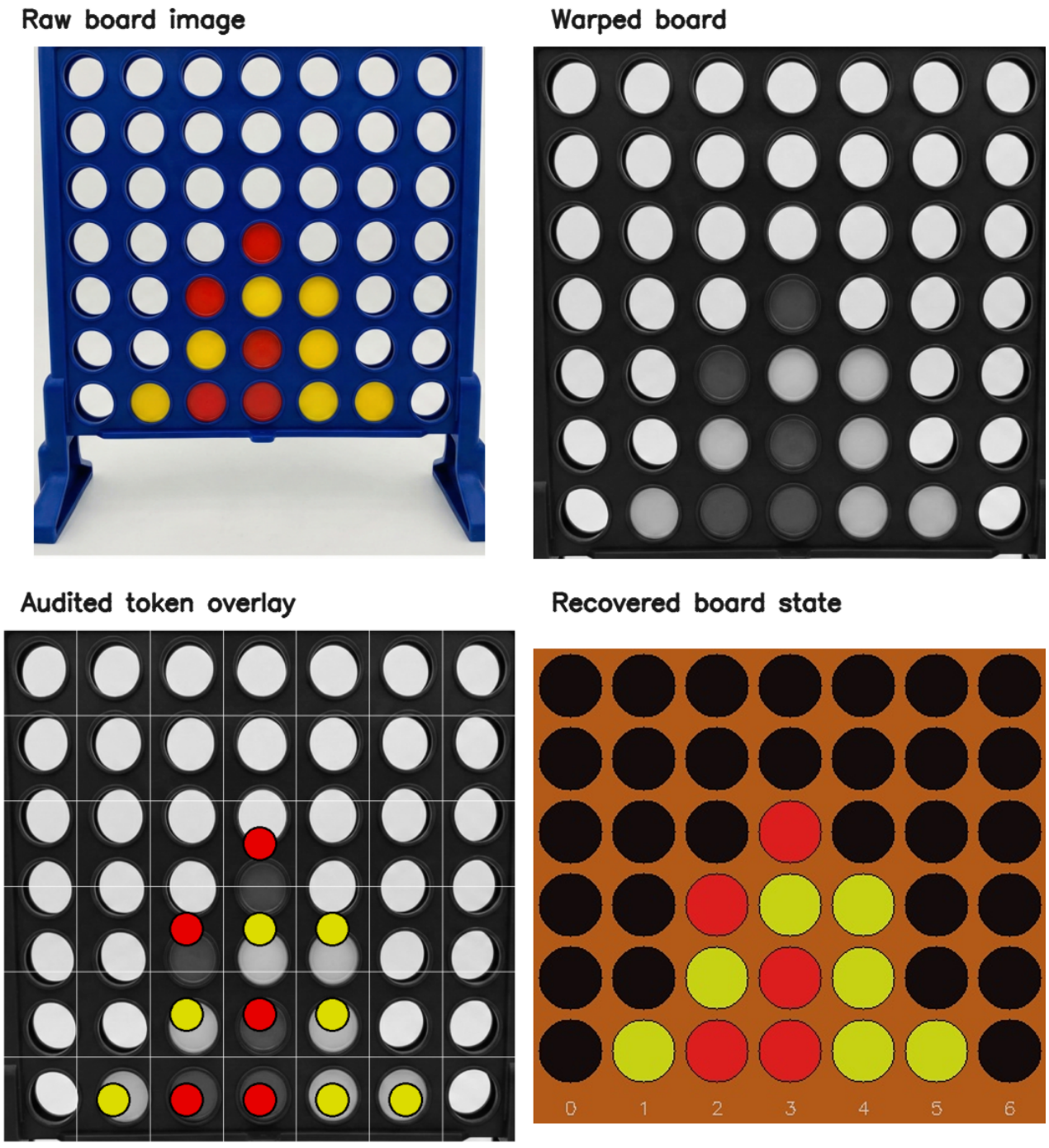



Fig. 3. Representative vision pipeline output from a single board position. The raw board image shows the starting scene, the warped board shows the geometry-normalized view used for cell analysis, the audited token overlay shows the interpreted token locations on that same warped image, and the recovered board state shows the discrete $6 \times 7$ representation used by the game logic.

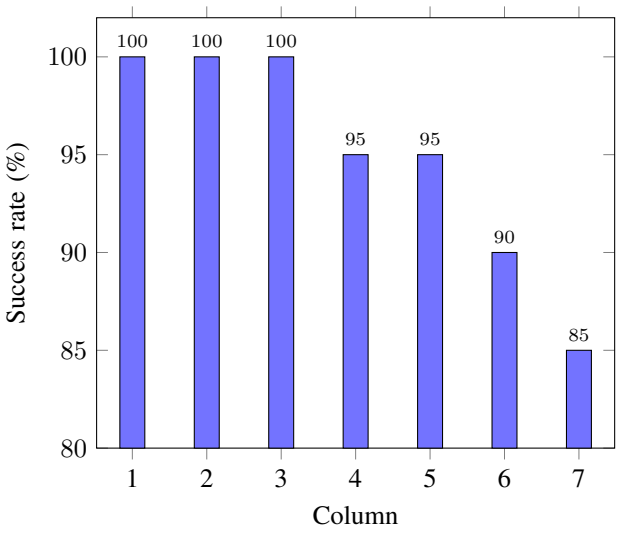



Fig. 4. Token-drop success by column. The monotonic falloff with travel distance shows that the dominant integrated failure mode is carriage positioning at the far columns, not token recognition.

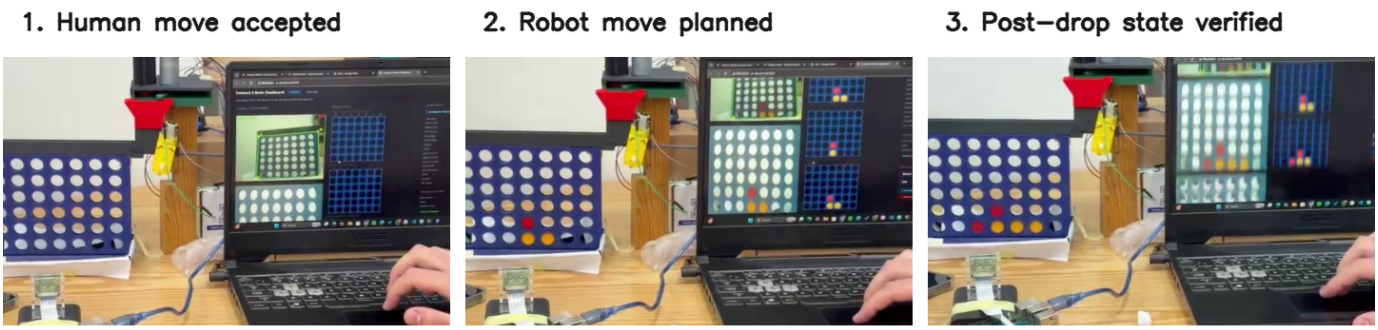



Fig. 5. System-level proof of closed-loop behavior from the final demo. A human move is accepted first, the robot plans from the trusted board, and the internal state is only advanced after the camera confirms the physical post-drop result.